

National Research University Higher School of Economics
International College of Economics and Finance (ICEF)

Graph-based Analysis of Currency Triangles in FX Markets

Detecting and Exploiting Triangular Arbitrage

*A course paper submitted in the first year
of the MSc programme*

Author: Nikolay Dvoryanchikov
Supervisor: Yaroslav Lyulko

Moscow, 2026

Contents

1	Introduction	1
2	Literature Review and Theoretical Background	2
2.1	Reinforcement learning and Markov decision processes	2
2.2	Continuous control: actor-critic methods and TD3	3
2.3	The foreign-exchange market and triangular arbitrage	3
2.4	The currency graph and graph attention	4
2.5	Literature review	5
3	Model and Methodology	7
3.1	The Markov decision process	7
3.2	State: the currency graph	8
3.3	Action and arbitrage execution	8
3.4	Transition and reward	10
3.5	The graph-attention encoder	10
4	Experimental Design	11
4.1	The synthetic market	11
4.2	Benchmarks and the oracle	12
4.3	Metrics	12
5	Results and Discussion	13
6	Conclusion, Limitations, and Future Work	18
A	Reproducibility and hyperparameters	21
B	Notation	22

1 Introduction

The foreign-exchange market moves trillions of dollars a day, and the banks and treasuries that operate in it end each day holding currency they never set out to own. Lending, hedging, and client flow leave every institution with a net position in each currency. That net position is its open currency position, and it matters: the institution gains or loses as exchange rates move against that position, and supervisors and internal mandates cap how large it may grow.

Keeping the open position where it belongs is a standing problem, not a single choice. Rates change with every quote, so a position that suits an institution one minute suits it less the next. Pulling it back into line means buying and selling currency, and each trade pays the gap between bid and ask, so frequent correction drains money even when each trade looks sound. The currencies move together as well, so a treasurer cannot handle one exposure at a time. And the treasurer commits to each decision before seeing the next move in rates.

A fixed rule struggles against a problem of this shape. The task asks instead for a way to decide, again and again and under uncertainty, how much exposure to carry in each currency while weighing expected return against the cost of trading and the limits on risk.

Problems with this structure, where an agent acts in sequence, pays for what it does, and measures success over a horizon instead of a single step, are the domain of reinforcement learning. An agent of this kind learns a policy from the outcomes of its own decisions and improves against an objective that reflects what a treasury values.

I propose an agent of this kind for the open currency position. It learns continuous control with TD3, an actor-critic algorithm in which a critic estimates the value of the current positions and an actor proposes how to change them. At each step the agent reads the market together with its own holdings and outputs a shift in exposure across the currencies. Its reward is the change in portfolio value after the cost of trading, so it learns to seek return while paying for every adjustment it makes.

Inside this broad task I target one concrete opportunity: triangular arbitrage. When the rates among three currencies fall out of step, converting from one currency through the other two and back returns more than it took to begin, a gain that holds until the rates realign. Spotting the inconsistency is arithmetic. Capturing it is the hard part, because a trader can run the loop only with currencies already in hand, so the profit depends on what the open position holds the moment the loop appears. Positioning for it means carrying exposure and paying spread in advance, with no promise the loop arrives before the rates close. Capturing arbitrage is therefore part of managing the open position: the same choices about what to hold, at what cost, under the same uncertainty. I ask whether my agent can learn to hold the right exposure and seize these loops as they form.

To do that, the agent needs a particular view of the market. A loop spans three currencies at once, so an agent that reads each currency on its own will miss it; the

signal lives in the connections between currencies. I therefore present the market as a graph, with currencies as nodes and exchange rates as the edges that join them, and I build the agent's state with a graph attention network. Attention lets the agent learn which connections to weigh, so it can recognise a forming loop and judge which currencies are worth holding to fund the loops that recur. With a graph-attention state, the agent sees the relationships that both the opportunity and the funding decision depend on.

This paper develops that agent and asks whether, with a graph-attention state, it can manage the open currency position by detecting and capturing triangular arbitrage.

2 Literature Review and Theoretical Background

This section establishes the ideas the project rests on and then places the work in its literature. The background comes first, in the order the agent uses the ideas: reinforcement learning, the TD3 algorithm, the foreign-exchange market and triangular arbitrage, and the graph attention network that draws them together. The literature review follows.

2.1 Reinforcement learning and Markov decision processes

Reinforcement learning studies an agent that learns to act by interacting with an environment [1]. At each step t the agent observes a *state* s_t , a description of the current situation; takes an *action* a_t ; and receives a scalar *reward* r_t that scores the immediate result. The environment then moves to a new state and the cycle repeats. The agent's behaviour is its *policy* π , a rule that maps a state to an action, and learning means improving that policy from experience alone, with no teacher supplying the correct action.

The agent does not maximise the next reward but the *return*, the discounted sum of all rewards from the current step onward,

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad \max_{\pi} \mathbb{E}_{\pi}[G_t], \quad (1)$$

where r_{t+k} is the reward k steps in the future and $\gamma \in [0, 1)$ is a *discount factor* that makes sooner rewards count more than later ones; a value of γ near 1 makes the agent far-sighted. This setup is a *Markov decision process* (MDP), specified by states, actions, a reward, the probabilities of moving from one state to the next, and γ . Its defining assumption, the *Markov property*, is that the next state depends only on the current state and action, not on the whole history. That assumption matters directly for me: the state must contain everything relevant to the decision, because a policy can be no better than the state it is handed. How I turn the market into such a state, the subject of the graph representation below, is therefore central rather than incidental.

In my project the mapping is concrete. The state is the current market together with the agent's holdings; the action adjusts the open currency position; the reward is the change in portfolio value after trading costs; and one step is one tick of market time. Maximising discounted return then means managing the position well across the whole horizon, not winning a single trade.

2.2 Continuous control: actor–critic methods and TD3

My actions are continuous: the agent sets real-valued adjustments to its positions rather than picking from a short list. This rules out methods that score every possible action and take the best, since there are infinitely many. Actor–critic methods avoid the problem by splitting the work. An *actor* proposes an action, and a *critic* estimates how good it is; the critic's estimate tells the actor which way to adjust.

I use Twin Delayed Deep Deterministic Policy Gradient (TD3) [2]. The actor μ_θ is a neural network with parameters θ that maps a state to one specific action. The critic Q_ϕ , with parameters ϕ , estimates the *value* of taking an action in a state, meaning the return expected from then on. The actor is trained to produce actions the critic rates highly, and the critic is trained so its estimates match the rewards actually received plus the value of the state that follows. TD3 sharpens an earlier algorithm, DDPG [3], which tends to overestimate values and train unstably. It adds three corrections, all of which I keep: it trains two critics and trusts the smaller estimate, which curbs systematic overestimation; it updates the actor less often than the critics, so the policy follows values only once they settle; and it adds small noise to the action when training the critic, so the actor cannot latch onto a sharp but spurious peak. Together these make a stable continuous-control algorithm, which is why I adopt TD3 over the less robust members of the same family [4].

2.3 The foreign-exchange market and triangular arbitrage

The agent acts in the spot foreign-exchange market, where one currency is exchanged for another at a quoted rate. Every pair carries two prices: a *bid*, the rate at which a dealer buys the base currency from you, and a higher *ask*, the rate at which the dealer sells it to you. The difference between them, the *spread*, is the cost of transacting, and it is paid on every leg of every trade. I write $R(i, j)$ for the executable rate that converts one unit of currency i into currency j after its spread is paid.

Not every pair is quoted directly. A missing rate can be implied from two others through a shared currency, giving a *cross-rate*, and the *no-arbitrage* condition requires the direct and implied rates to agree. *Triangular arbitrage* is the case where that condition breaks across three currencies. Taking one unit of currency a to b , then to c , then back to a leaves the product of the three rates,

$$\phi = R(a, b) R(b, c) R(c, a), \quad (2)$$

which I call the *cycle factor*: the multiple of the starting amount held after going

around the loop. Because each conversion already pays its spread, the loop turns a profit exactly when $\phi > 1$, and that profit is $\phi - 1$ per unit committed; when $\phi \leq 1$ the loop loses money and is left alone. In real markets such loops are small and disappear within moments [5]. This one number, computed over a three-currency loop, is the signal the agent must learn to read and act on.

2.4 The currency graph and graph attention

The preceding ideas leave a gap to close. The agent needs a state that captures the market, and the market’s information is relational: a rate links two currencies, consistency ties rates to each other, and an arbitrage loop spans three currencies at once. A flat list of numbers can store these quantities but discards how they connect, which is where the signal lives. I need a representation that keeps the connections, and then a way to turn it into the state vector the agent reads.

The representation is a *graph*. A graph $G = (V, E)$ is a set of *nodes* V and a set of *edges* E , each edge joining two nodes to record a relationship. When nodes or edges carry *features*, vectors of numbers describing them, the graph is *attributed*; when an edge points from one node to another, it is *directed*. A graph is the natural container when the relationships, rather than the entities on their own, carry the information.

For foreign exchange the mapping is immediate. Each currency is a node, and each executable rate is a directed edge from one currency to another, since converting i into j differs from the reverse. The four currencies of this study give four nodes and twelve directed edges, one per ordered pair (Figure 1). The features hold what the agent needs each tick: a node carries the currency’s current and previous portfolio weight, and an edge carries the rate, its spread, its recent volatility and momentum, and the arbitrage signal for loops through it. The triangular loop of the previous subsection is now a three-edge *cycle* in this graph, and its cycle factor ϕ is a property of that cycle. The structure of the market, and the signal hidden in it, are now explicit in the representation.

A graph is still not the vector a policy can read, and flattening it would discard the structure just preserved. A *graph neural network* turns the graph into vectors while respecting its connections, through *message passing* [6, 7]. Each node starts from its feature vector and refines it over rounds; in each round a node gathers its neighbours’ current vectors, combines them into one message, and updates its own vector from that message. With $h_i^{(l)}$ the vector of node i after round l ,

$$h_i^{(l+1)} = \text{update}\left(h_i^{(l)}, \text{aggregate}\{h_j^{(l)} : j \text{ adjacent to } i\}\right), \quad (3)$$

where *aggregate* fuses the neighbours and *update* folds the result back into the node. Stacking L such rounds carries information up to L steps across the graph, so each currency’s vector comes to reflect the currencies it connects to, not itself alone.

How a node weights its neighbours decides what that message carries, and for me it is the whole point. The simplest networks weight every neighbour equally,

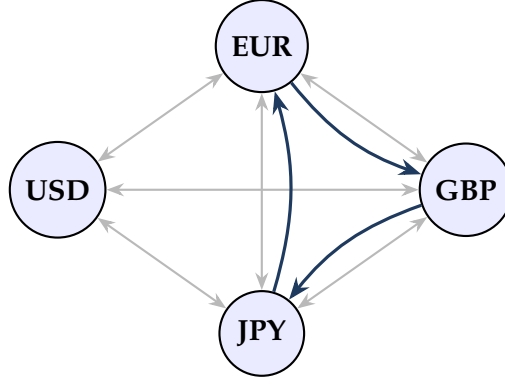


Figure 1: The currency graph as a directed graph for $N = 4$. Every pair carries a rate in each direction, drawn as a double-headed edge, for $N(N - 1) = 12$ directed edges in all; the accent path $\text{EUR} \rightarrow \text{GBP} \rightarrow \text{JPY} \rightarrow \text{EUR}$ is one of the eight directed three-cycles, a triangular-arbitrage loop that pays when its cycle factor ϕ of (2) exceeds one.

or by a fixed rule based on how many connections each node has [8]. My signal is the opposite of uniform: at any tick the three edges of a mispriced loop matter and the rest are noise, so equal weighting averages the few informative edges in with many uninformative ones and dilutes exactly what the agent should see. A *graph attention network* (GAT) removes this limitation by learning the weights and letting them depend on the content of the nodes [9, 10]. For each neighbour j of node i it computes an *attention weight* $\alpha_{ij} \in [0, 1]$ saying how much j should count for i , then updates i as a weighted sum,

$$\alpha_{ij} = \text{softmax}_j (\text{score}(h_i, h_j)), \quad h'_i = \sigma \left(\sum_j \alpha_{ij} W h_j \right), \quad (4)$$

where *score* is a small learnable function comparing two nodes, softmax_j normalises the weights over i 's neighbours so they sum to one, W is a learned matrix that transforms each neighbour's vector, and σ is a non-linearity. Several independent weightings, called *attention heads*, run in parallel, so a node can attend to more than one relationship at once.

Because the deciding quantities in my market sit on the edges, I feed the edge features into *score*: the weight a currency places on a neighbour then depends on the rate, spread, and arbitrage signal that connect them, and the attention can settle on the edges of a loop that is currently live. The network returns one refined vector per currency, and these are concatenated into the single state vector that the TD3 actor and critic consume. This is the state the MDP demanded: the graph holds the market's structure, attention keeps the part that matters, and the result is what the agent acts on.

2.5 Literature review

I now look at how others have approached the parts of this problem, what they found, and where my plan sits among them. Three lines of work bear on it directly:

reinforcement learning for trading and portfolio control, graph models of financial markets, and the study of arbitrage.

Reinforcement learning reached trading through direct, or recurrent, reinforcement learning. Moody and Saffell trained a policy online to maximise the differential Sharpe ratio with transaction costs written into the objective, and showed two things that still hold: optimising the policy directly works better than learning a value function and then acting on it, and realistic costs change the shape of the policy rather than trimming its edges [11]. Deng et al. kept that direct-policy idea but learned the input features with a deep network instead of choosing them by hand, and improved trading on real futures and stocks [12]. Jiang et al. turned to allocation across many assets: their agent scores each asset with an identical sub-network and carries a memory of the previous weights so that rebalancing has a price, and in a cryptocurrency backtest it multiplied its capital several times over fifty days even under a 0.25% commission [13]. Liang et al. set the continuous-control algorithms side by side on Chinese equities, reported that a plain policy gradient trained more reliably than DDPG or PPO, and raised return and Sharpe with adversarial price perturbations [14]; the fragility they saw in DDPG is the weakness the TD3 stabilisers of Section 2.2 were built to fix. Tsai and Wang carried the approach to currencies, turning each price window into a Gramian Angular Field image, driving a Sure-Fire statistical-arbitrage rule, and comparing DQN against PPO on three FX pairs [15]. These systems share one trait that bears on me: each reads a flat or per-asset state, so the relations among currencies stay implicit and arbitrage never reaches the action. I keep their durable lessons, a cost-aware risk-adjusted reward and a continuous policy, and change exactly that state.

The missing relations are what graph models add. Feng et al. ranked stocks with a temporal model joined to a graph convolution over relations drawn from sector membership and corporate links, and found that the relations improved both the ranking and its simulated return over models that ignore them [16]. Shi et al. moved the graph inside the reinforcement-learning loop: their GPM agent extracts inter-asset relations with a relational graph convolution, combines them with multi-scale temporal features, and outperforms non-relational portfolio agents [17], and graph-based reinforcement learning for portfolios has stayed active since. The pattern is consistent, a relational state helps a learning trader, but two gaps remain for me. These studies live in equities, and their edges stand for similarity or economic kinship, whereas an FX edge is an executable conversion rate, and a closed loop of such rates is exactly where arbitrage appears, the structure I set up in Section 2.4.

Arbitrage itself has been studied mostly as a detection problem. The statistical-arbitrage tradition descends from Engle and Granger, who showed how to fix a stationary combination of prices and trade its deviations, a structure the modeller specifies rather than learns [18]. Triangular arbitrage is the deterministic, cross-sectional version, and the evidence on it is cautionary: measuring executable spot-FX quotes, Fenn et al. found gaps below one basis point that close in under a second and have grown scarcer as trading electronified, so winning enough of them to profit is

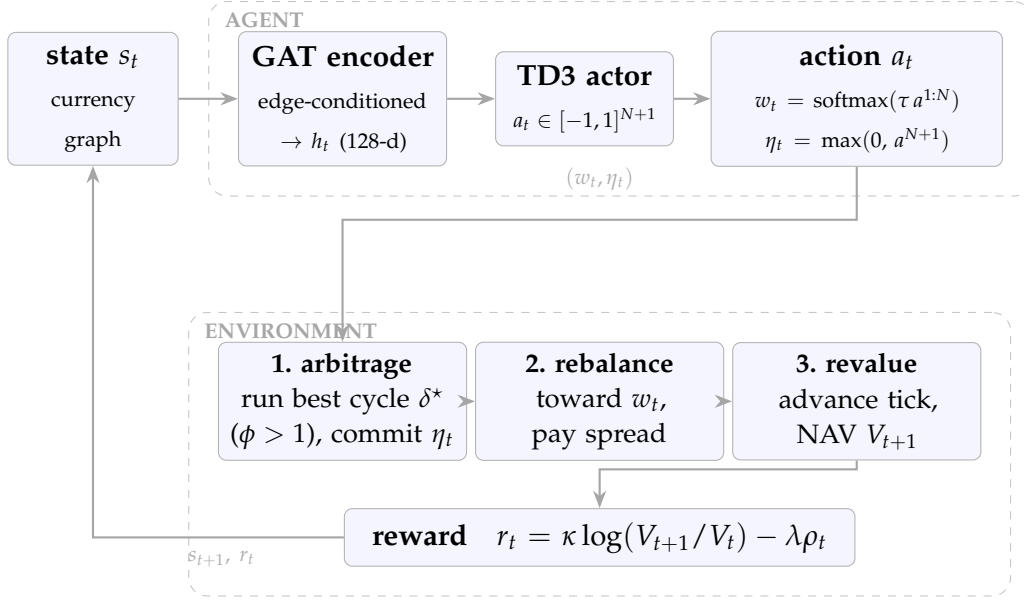


Figure 2: The system as one decision loop. The currency-graph state s_t is encoded by the graph attention network into a vector h_t , and the TD3 actor maps it to an action that splits into a target allocation w_t and an arbitrage intensity η_t . The environment runs the best profitable cycle δ^* at intensity η_t , rebalances toward w_t while paying the spread, advances one tick and revalues to V_{t+1} , and returns the next state s_{t+1} and the reward r_t .

unrealistic [5]. Graph learning has reached this corner too: Hong and Klabjan model currencies as nodes and rates as edges, forecast the rates with a spatiotemporal graph network, and choose statistical-arbitrage trades by constrained optimisation that accounts for execution delay, reporting a large gain in information ratio over their benchmarks [19]. This body of work detects the inconsistency, or predicts the prices around it; it does not learn a policy that decides how much to commit and which currencies to hold in order to capture it.

My plan sits where these lines meet, and it differs from each in a definite way. Where reinforcement-learning traders read a flat state, I read the currency graph of Section 2.4; where graph models in finance encode similarity among equities, my edges are executable FX rates whose cycles carry the arbitrage; and where the arbitrage literature detects or forecasts, my agent acts, allocating exposure and capturing the loops the graph reveals through the TD3 controller of Section 2.2. How I test this plan, and against what reference, belongs to the experimental sections that follow, and I settle it there rather than prejudice it here.

3 Model and Methodology

3.1 The Markov decision process

Single-account FX portfolio control is a Markov decision process, the tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$: a state space $\mathcal{S}$ of market descriptions paired with the agent's

holdings, an action space $\mathcal{A} = [-1, 1]^{N+1}$, a transition kernel $P(s_{t+1} \mid s_t, a_t)$ set jointly by the market's evolution and the agent's own trades, a scalar reward r_t , and a discount $\gamma \in [0, 1)$. The agent learns a policy that maximises the expected discounted return of (1), and Figure 2 shows the loop. The rest of this section fixes each component: the holdings and their value, the state s_t , the action a_t , the dynamics P , and the reward r_t . The full notation is collected in Appendix B.

The agent holds non-negative balances $b_t^i \geq 0$ of each currency i , with $\mathcal{C} = \{1, \dots, N\}$ the traded currencies, USD the numéraire, and $N = 4$ (USD, EUR, GBP, JPY) in the experiments. Currencies trade at executable rates quoted with a bid $\beta_t(i, j)$ and a higher ask $\alpha_t(i, j)$ for turning i into j . Selling i realises the lower bid, so converting an amount x returns $\text{conv}_t(x, i, j) = x \beta_t(i, j)$, and the gap up to α_t is the cost the trade pays. Valuing each holding in USD and summing gives the net asset value (NAV), and the share each currency takes of it is its weight,

$$v_t^i = \begin{cases} b_t^i, & i = \text{USD}, \\ b_t^i \beta_t(i, \text{USD}), & i \neq \text{USD}, \end{cases} \quad V_t = \sum_{i \in \mathcal{C}} v_t^i, \quad \omega_t^i = \frac{v_t^i}{V_t}, \quad (5)$$

with $\omega_t \in \Delta^{N-1}$ on the simplex and $V_0 = \$10,000$. Three of these quantities are load-bearing later. The value v_t^i reappears when the environment chooses which arbitrage to run and when the reward measures growth. The weight ω_t^i enters the state and the trading cost, and the bid β_t that values the book also prices every leg of an arbitrage loop.

3.2 State: the currency graph

The state s_t pairs the agent's holdings with an attributed directed graph G_t . Its node set V is the N currencies and its edge set $E = \{(i, j) : i \neq j\}$ is every ordered pair, $N(N-1) = 12$ for $N = 4$ (Figure 1), so the topology is fixed once by the currency set and never inferred. Only the node features X_t and edge features Z_t change each tick. Each node i carries a three-vector $x_t^i \in \mathbb{R}^3$: its current weight ω_t^i , its weight one tick earlier ω_{t-1}^i , and the best arbitrage rate g_t^i it can fund. Each edge (i, j) carries a five-vector $z_t^{ij} \in \mathbb{R}^5$: the log mid-rate, the relative spread $s_t^{ij} = (\alpha_t - \beta_t)/m_t^{ij}$, the volatility and the momentum of the rate over a trailing window of W ticks, and the arbitrage signal q_t^{ij} . Table 1 lists them. The two arbitrage features g_t^i and q_t^{ij} come from the cycle factor defined two paragraphs below.

3.3 Action and arbitrage execution

The action carries the agent's two intentions in one vector $\mathbf{a}_t \in [-1, 1]^{N+1}$, its first N entries a target allocation and its last an arbitrage intensity, each mapped to a usable range by a softmax and a rectifier,

$$\mathbf{w}_t = \text{softmax}(\tau \mathbf{a}_{t,1:N}) \in \Delta^{N-1}, \quad \eta_t = \max(0, a_{t,N+1}) \in [0, 1]. \quad (6)$$

Table 1: State features. Node features come from the agent’s own bookkeeping, and edge features are market-derived over the window of length W .

Symbol	Name	Description
<i>Node features $\mathbf{x}_t^i \in \mathbb{R}^3$</i>		
ω_t^i	current weight	current portfolio weight of currency i
ω_{t-1}^i	previous weight	weight at the previous decision
g_t^i	best arb. rate	best executable cycle return fundable <i>from</i> i
<i>Edge features $\mathbf{z}_t^{ij} \in \mathbb{R}^5$</i>		
$\log m_t^{ij}$	log mid-rate	log of the mid conversion rate $i \rightarrow j$
s_t^{ij}	spread	relative spread $(\alpha - \beta)/m$
σ_t^{ij}	volatility	std. of log-returns of m^{ij} over the window
μ_t^{ij}	momentum	window return $m_t^{ij}/m_{t-W+1}^{ij} - 1$
q_t^{ij}	arb. signal	best cycle return over triangles through (i, j)

The softmax keeps $\mathbf{w}_t$ on the simplex Δ^{N-1} , a long-only allocation summing to one. The temperature τ matters because each entry $a_{t,k}$ lies in $[-1, 1]$: a softmax over scores spanning only two units returns weights close to uniform, so an unscaled action could never concentrate capital. Multiplying by $\tau > 1$ widens that range, letting the output run from near-uniform to nearly a single currency, so τ sets the strongest position the agent can request, and I fix it in advance. The rectifier makes η_t non-negative, so $\eta_t = 0$ is a clean “no arbitrage” choice. The target $\mathbf{w}_t$ is distinct from the current weights ω_t , and the difference between them is what the agent trades to close.

Arbitrage lives on the cycles of the graph and is priced by the same bids that valued the book. I enumerate the directed triangles once into a set $\mathcal{T}$ of three-cycles $\delta = (i \rightarrow j \rightarrow k \rightarrow i)$ over distinct currencies, eight of them for $N = 4$, and one unit carried around δ returns the cycle factor $\phi_t(\delta) = \beta_t(i, j) \beta_t(j, k) \beta_t(k, i)$. Each leg has already paid its spread inside β_t , so the loop profits exactly when $\phi_t(\delta) > 1$, earning $\phi_t(\delta) - 1$ per unit committed. This factor defines the two arbitrage features of the state,

$$q_t^{ij} = \max\left(0, \max_{\delta \ni (i,j)} \phi_t(\delta) - 1\right), \quad g_t^i = \max\left(0, \max_{\delta \ni i} \phi_t(\delta) - 1\right), \quad (7)$$

where $\delta \ni (i, j)$ ranges over triangles using edge (i, j) and $\delta \ni i$ over those touching currency i : q_t^{ij} is how much the best loop through that edge pays beyond breaking even, and g_t^i how much the best loop currency i could fund pays, each zero when no loop there is profitable. To act, the environment takes the single profitable loop with the largest dollar gain, the per-unit profit times the capital available to fund it,

$$\delta_t^* = \arg \max_{\delta \in \mathcal{T}: \phi_t(\delta) > 1} (\phi_t(\delta) - 1) \sum_{i \in \delta} v_t^i, \quad (8)$$

and runs δ_t^* from each currency i in it that the agent holds, committing $\eta_t b_t^i$ of that balance. Each leg round-trips to its funding currency, so the weights barely move and the gain accrues to the next NAV. Because a loop can be funded only from currencies already held, the allocation $\mathbf{w}_t$ decides which arbitrage is even reachable. When no loop clears one, the agent does no arbitrage.

3.4 Transition and reward

One step runs in a fixed order: the agent computes $\mathbf{w}_t$ and η_t , the environment executes δ_t^* at intensity η_t , then rebalances toward $\mathbf{w}_t$ by selling before buying and paying the spread on each leg, and finally advances to $t+1$ and revalues the holdings at the new prices to give V_{t+1} . With $\omega_{t,\text{pre}}^i$ and $\omega_{t,\text{post}}^i$ the weights just before and after rebalancing, the turnover

$$\rho_t = \frac{1}{2} \sum_{i \in \mathcal{C}} |\omega_{t,\text{post}}^i - \omega_{t,\text{pre}}^i| \quad (9)$$

is the fraction of the book that changed hands, halved because a dollar sold from one currency is bought into another and would otherwise count twice, so $\rho_t = 0$ is no trade and $\rho_t = 1$ a full turnover. The reward then measures growth in the same NAV while charging that churn,

$$r_t = \kappa \log \frac{V_{t+1}}{V_t} - \lambda \rho_t, \quad \kappa, \lambda > 0. \quad (10)$$

The spread has already paid the real transaction cost inside a smaller V_{t+1} , so λ only discourages needless trading rather than standing in for the cost, and κ lifts the small per-tick log-return to a usable scale. An episode ends if the drawdown from the running NAV peak exceeds a threshold, and otherwise truncates when the data end.

3.5 The graph-attention encoder

The graph G_t must finally become the fixed-length vector the policy reads, which is the job of the encoder, a two-layer edge-conditioned graph attention network. It is edge-conditioned because the attention score of Section 2.4 takes in the edge features z_t^{ij} , so a currency weights a neighbour by the rate, spread, and arbitrage signal that connect them. The first layer maps each node's three features to a sixteen-dimensional representation under four attention heads, whose concatenation gives sixty-four dimensions per node. The second maps that to a thirty-two-dimensional embedding per currency, both with LeakyReLU. Stacking the four embeddings and applying layer normalisation produces the 128-dimensional state vector that the TD3 actor [2] and its twin critics, each a small multilayer perceptron, take as input. The fixed edge index is built once and reused, so encoding a state is a single forward pass. The agent is implemented with Stable-Baselines3 [20] and PyTorch Geometric [21] within a Gymnasium [22] environment.

The exact network widths, random seeds, and training length are listed in Appendix A and the notation in Appendix B. Full seeding and a stored fingerprint of the data-generating configuration make every run reproducible. With the model and its environment fixed, the next section turns to how they are evaluated.

4 Experimental Design

The experiments are designed to evaluate representation learning under controlled conditions rather than to estimate live trading profitability. Because the synthetic market contains a known quantity of injected arbitrage and admits an oracle upper bound, the central questions of whether the agent detects and exploits the structure, what makes the signal learnable, and where any advantage originates can be examined against ground truth rather than inferred from noisy returns.

4.1 The synthetic market

I generate the market myself so that I know exactly how much arbitrage it contains and when. The base market is built to contain none. Each non-USD currency follows its own random price process in dollars (a geometric Brownian motion), the dollar is held at one, and every quoted rate is read off these values, so the rate from i to j is simply their ratio. Because all rates come from the same few numbers, the cross-rates stay mutually consistent and no triangle pays.

Arbitrage is then added on purpose. In randomly placed windows I push a few cross-rates off their consistent values, opening loops that pay until the push decays back to zero, and now and then two legs are pushed at once so that the pure-cross EUR/GBP/JPY triangle can become the most profitable loop. A small fixed half-spread on every quote turns the mid-rates into a bid and an ask, so each trade pays a realistic cost. Training and testing draw independent paths from this generator, so the agent is never tested on the path it trained on, and a harder regime-shift draw reverses the drift and raises volatility to check that what the agent learned is not tied to one regime. Path length, window, and seeds are listed in Appendix A.

One detail about the inputs matters. The features the agent reads span very different magnitudes: the log rate varies by a few units, the arbitrage signal by about 10^{-4} , four orders of magnitude smaller. Since the attention layer mixes features through sums and inner products, the rate level would otherwise swamp the small but decisive channels (arbitrage, momentum, spread), so I multiply each feature by a fixed constant that brings every channel to roughly unit scale. The constants are fixed rather than estimated from the data, which keeps the scaling identical on the training and test paths.

4.2 Benchmarks and the oracle

The agent is measured against five simple policies, each run through the same dynamics and costs so the comparison is fair. The *cash* policy holds dollars throughout and never trades, the flat do-nothing line. *Equal-weight* keeps a quarter of the book in each currency, rebalancing every tick to hold that split. *Buy-and-hold* sets the same equal split once and then leaves it untouched, paying no further cost. The *random* policy draws a fresh allocation each tick from a Dirichlet distribution, so it trades constantly and exposes what pure churn costs. *Momentum* tilts toward the currencies whose recent return against the dollar has been positive, the simplest directional rule. None of the five attempts arbitrage, so together they measure what allocation alone can earn under these costs.

The *oracle* is a reference of a different kind, not a policy I expect to beat but a ceiling on how much arbitrage a diversified book can capture. It does two things every tick. It holds a fixed, evenly diversified book, an equal quarter in each currency, so that whichever cycle turns profitable it always holds the currencies needed to fund it. And it sets its arbitrage intensity to the maximum, so the environment runs the single most profitable cycle then available at full size, through the same gated execution and the same spreads every other policy faces. The oracle therefore never hesitates and never mis-sizes: it takes all the arbitrage a diversified book can reach, tick after tick.

This makes it a yardstick rather than a competitor. It is idealised in the same way as the rest of the study, executing at the observed quotes with no latency or slippage, and it is not a strategy anyone would run, since it commits fully on every opportunity with no regard for risk or for the chance that the opportunity reverses. Two limits are worth keeping in mind. Its book is diversified but fixed rather than chosen, so it bounds arbitrage capture only among diversified books and not absolutely: a policy that tilts its holdings toward the currencies that fund the most frequent cycles could in principle take more. And because that fixed book also carries open currency exposure, the oracle's return is its arbitrage capture together with whatever its holdings gain or lose on direction. I report it for a single purpose, to mark how much injected arbitrage is there to be had, so that the agent's capture can be read against a known ceiling rather than against zero.

4.3 Metrics

A fair comparison needs every policy judged on the same path, under the same costs, and by the same yardsticks, so this subsection fixes those yardsticks before the results. They serve two jobs. The first is plain ranking, by what each policy earns and the risk it takes to earn it. The second matters more here: separating where a policy's return comes from, because a single return number cannot say whether an edge is arbitrage capture or directional allocation, the question the paper turns on.

For ranking I report the usual figures. Total return says whether a policy made

Table 2: Test-path performance (in-distribution). *arb* and *alloc* are the arbitrage and allocation components of P&L (USD on a \$10,000 initial NAV).

Policy	Return	arb P&L	alloc P&L	Sharpe	MaxDD
Cash	0.0%	0	0	0.00	0.0%
Equal-weight	−2.6%	0	−255	−0.74	2.6%
Buy-and-hold	−2.5%	0	−251	−0.73	2.6%
Random	−6.3%	0	−625	−1.45	6.3%
Momentum	1.1%	0	112	0.17	2.3%
Arbitrage (oracle)	21.7%	2464	−296	2.58	—
TD3+GAT	28.2%	3213	−394	2.30	1.5%

money. The annualised Sharpe ratio divides that return by its volatility. Maximum drawdown records the worst peak-to-trough fall.

Two further instruments, built for this study, do the explaining. A profit-and-loss decomposition splits each policy’s total into an arbitrage part, summed from the per-step arbitrage profit, and an allocation part that is everything else, and so answers directly whether an edge comes from capturing arbitrage or from directional allocation. A per-triangle breakdown records, for each of the eight cycles, how often it is available, how often it is the most profitable, how often the agent funds it, and the profit it returns, and so shows which opportunities the agent takes and which it leaves.

Two caveats travel with the figures. Annualising on tick data is a convention, so the absolute Sharpe values rank policies within this study but do not compare to calendar-time figures. And every number comes from a single train/test pair, so the magnitudes are preliminary and no significance is claimed before a multi-seed study.

Table 2 reports these metrics for the agent against the benchmarks and the oracle on the in-distribution test path. Section 5 discusses it in full, traces the net-asset-value paths in Figure 3, and breaks the arbitrage down per triangle in Table 3, showing where the agent’s profit is earned and where it is missed.

5 Results and Discussion

In the reported train/test realisation the GAT+TD3 agent attains the highest total return among all policies evaluated. On the in-distribution test path it returns +28.2% at an annualised Sharpe of 2.30 and a maximum drawdown of 1.5%, while the simple benchmark policies are flat or negative: cash is flat, equal-weight and buy-and-hold lose about 2.5%, the random policy loses 6.3%, and momentum gains only 1.1% (Table 2). The agent’s return also exceeds that of the arbitrage oracle (+21.7%), for a reason examined below, and the same ordering holds on the regime-shift path (agent +21.1%, oracle +18.3%), which indicates that the outcome is not an artefact

Table 3: Per-triangle arbitrage capture (test path). “best” counts ticks where the triangle has the highest cycle factor; “exec.” counts ticks the agent funds it; profit is in USD.

Triangle	available	best	exec.	profit (\$)
EUR/GBP/JPY (pure-cross)	275	45	171	1861
EUR/GBP/USD	190	126	83	736
EUR/JPY/USD	110	80	64	494
GBP/JPY/USD	134	80	13	123

of the in-distribution trend.

Figure 3 shows the corresponding net-asset-value trajectories and the ranking of policies by annualised Sharpe. The agent and the oracle separate clearly and early from the benchmark policies, which remain close to the initial capital throughout, and the agent tracks above the oracle for most of the path. The trajectory is close to monotone, with drawdowns small relative to the cumulative gain, consistent with an advantage driven by the repeated capture of small, near-riskless opportunities rather than by directional exposure.

The same picture holds out of distribution. Figure 4 shows the regime-shift path, where the base drift is reversed and volatility inflated. The ordering of policies is preserved, with the agent and the oracle ahead of the benchmark policies, consistent with an advantage that derives from arbitrage capture rather than from the direction of the underlying trend.

The per-triangle breakdown in Table 3 shows where that capture happens. It funds every one of the 331 profitable windows on the test path, and its mean allocation, approximately $[\text{USD}, \text{EUR}, \text{GBP}, \text{JPY}] \approx [0.01, 0.08, 0.33, 0.58]$, is concentrated in GBP and JPY with very little USD. This positioning is not incidental: triangular arbitrage can only be funded from currencies the agent holds, and the most frequently profitable opportunity on this path is the pure-cross EUR/GBP/JPY triangle. By holding GBP and JPY the agent funds those cycles with the most capital, which makes the pure-cross triangle its largest single source of profit and explains why it exceeds the oracle: the oracle funds every cycle from a flat diversified book, whereas the agent over-weights the currencies that fund the most frequent opportunity. The same mechanism explains the one under-served triangle, GBP/JPY/USD, which requires the USD inventory the agent has chosen to shed.

Decomposing the return into its sources confirms where the edge lies and gives the central interpretive result. In the in-distribution realisation the agent’s total of +\$2,820 comprises +\$3,213 of arbitrage profit and −\$394 of allocation residual, and on the regime-shift path the corresponding figures are +\$2,105 = +\$3,116 − \$1,011. In every setting the arbitrage component is strongly positive and the allocation residual is negative, and it is more negative under the regime shift, where the directional book is on the wrong side of the trend. The agent’s advantage is therefore attributable to structure detection and arbitrage funding rather than to directional

forecasting. The allocation it learns is a device for funding arbitrage that, after costs, slightly detracts from return.

Taken together, the results answer the question posed in the introduction. The agent does learn to detect and exploit the injected triangular arbitrage, executing every profitable window, outperforming the simple benchmark policies, and capturing the majority of the oracle-bounded opportunity, including on the out-of-distribution path. The decomposition shows that the learned advantage is attributable to structure detection and the inventory positioning that funds it, not to reliable directional FX forecasting, whose contribution is negative.

Although the evidence is limited to the controlled synthetic environment, it is informative. It establishes that a graph-attention state representation can allow a reinforcement-learning agent to learn a structural market signal and to position itself to act on it. These findings support the representation-learning idea on its own terms and motivate a route toward decision support for open-currency-position management, in which a graph-based agent could assist in allocating and hedging exposure under transaction costs, once the method is taken to real data.

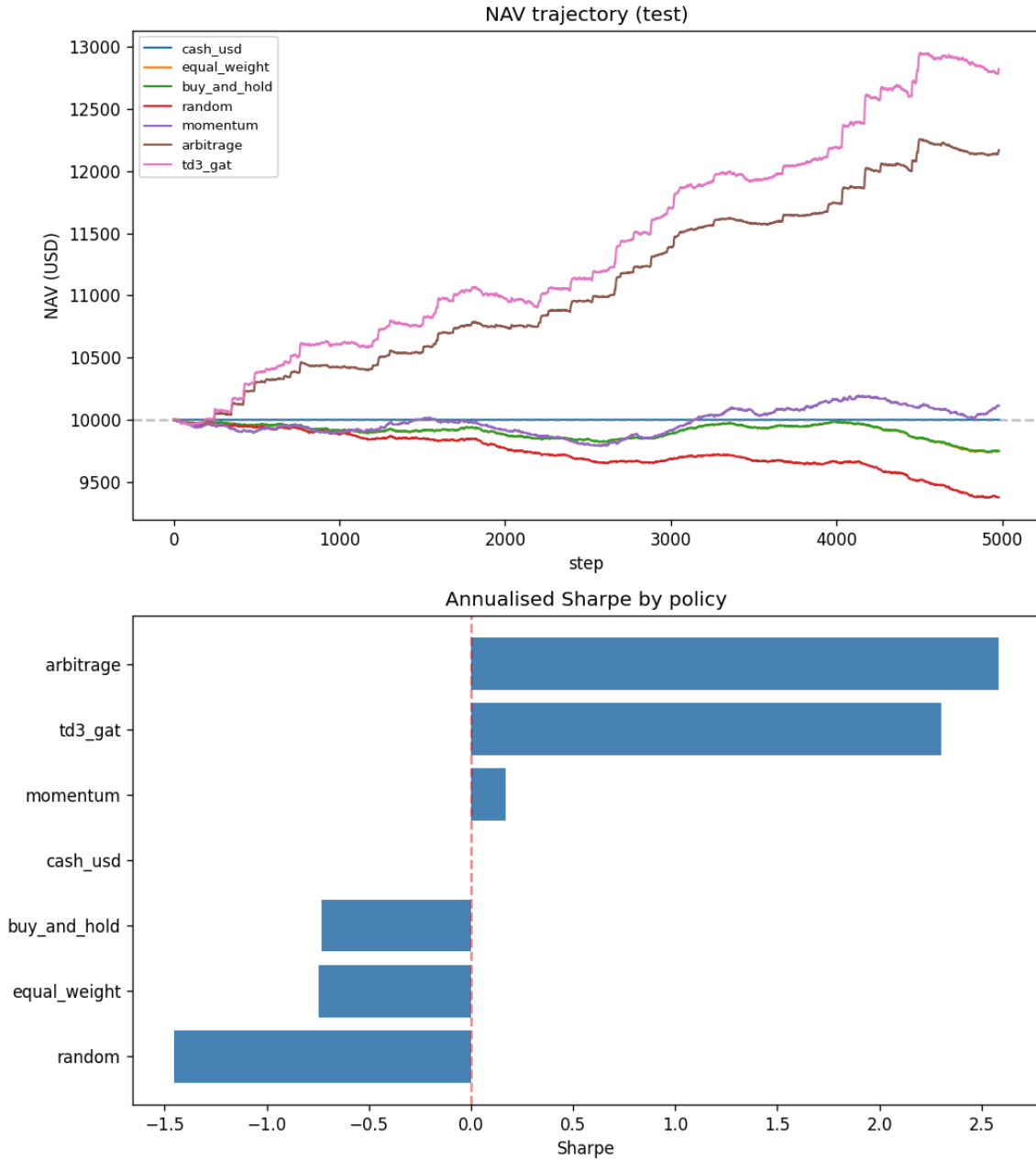


Figure 3: In-distribution test path. Top: NAV trajectories for the GAT+TD3 agent, the arbitrage oracle, and the benchmark policies. Bottom: policies ranked by annualised Sharpe. The agent and oracle separate from the benchmark policies early and remain ahead throughout.

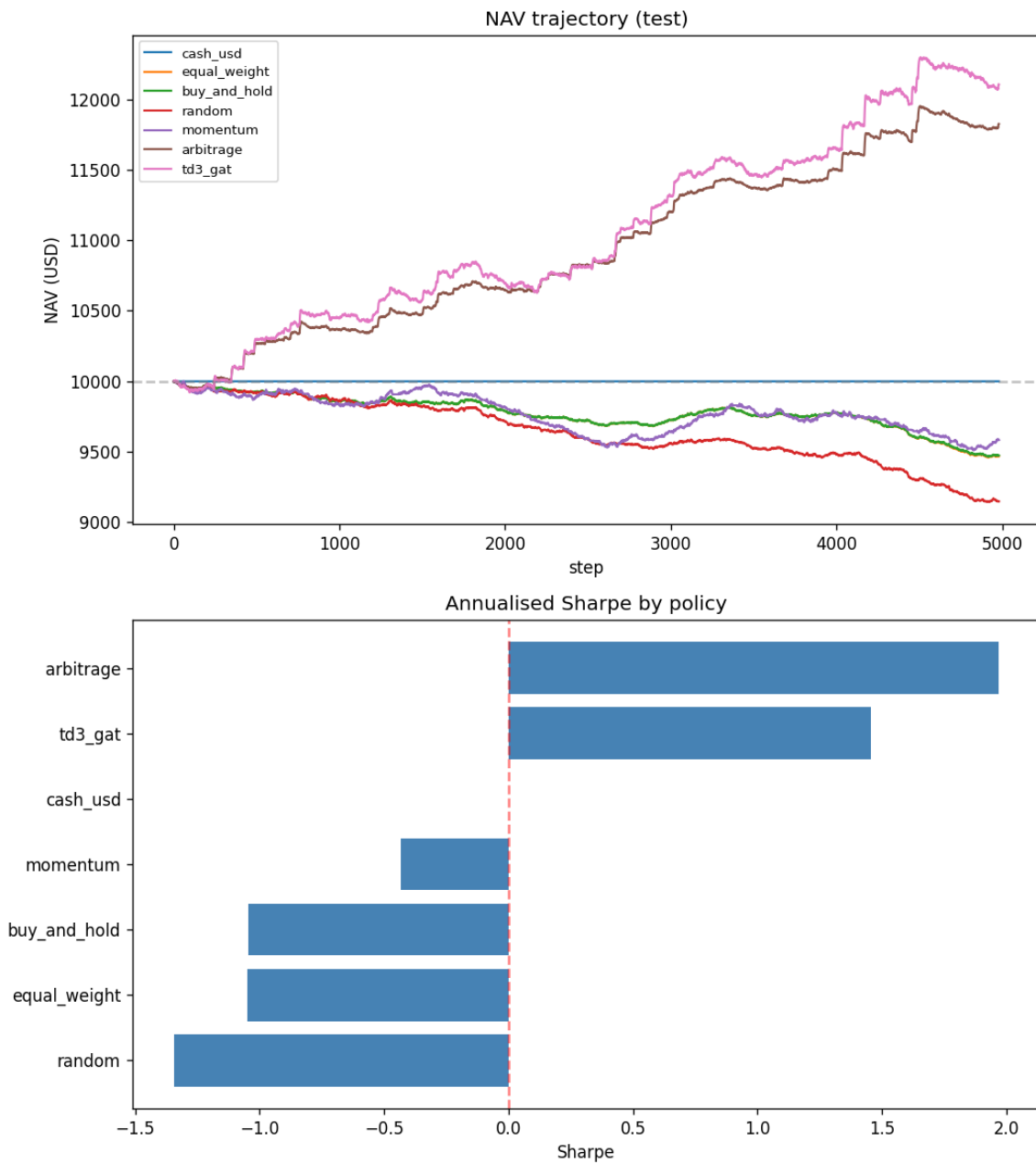


Figure 4: Regime-shift (out-of-distribution) test path. Top: NAV trajectories. Bottom: policies ranked by annualised Sharpe. The ordering matches the in-distribution path of Figure 3.

6 Conclusion, Limitations, and Future Work

This paper asked whether encoding the FX market as a currency graph and using graph attention as the state representation of a reinforcement-learning agent enables that agent to detect and exploit triangular arbitrage. Within the controlled synthetic environment the answer is affirmative: a TD3 agent with a graph-attention state learns to recognise the injected arbitrage and to position its holdings so as to capture it.

The main findings are consistent across the in-distribution and regime-shift paths, and the net-asset-value trajectories of Figures 3 and 4 show them at a glance. In both settings the agent climbs steadily ahead of the simple benchmark policies, which stay near their starting capital, and remains above the arbitrage oracle. It funds every profitable window and concentrates its inventory in the currencies that fund the most frequent opportunity. And the profit-and-loss decomposition attributes the agent’s advantage to arbitrage capture and inventory positioning rather than to directional forecasting.

The limitations are matters of scope, and should be kept clearly in view. The evidence comes from a synthetic environment in which the injected arbitrage is far larger and more persistent than in real foreign-exchange markets, where triangular mispricings are smaller than a basis point and gone within a second. Execution is idealised: trades fill at the observed quote, with no latency, no slippage, and a single constant spread standing in for the full cost of trading. The market itself is deliberately narrow, four major currencies driven by a controlled generator rather than real tick data. For these reasons the returns reported here are properties of the testbed, not evidence of live trading profitability.

Several directions follow. A controlled ablation comparing the graph-attention encoder against a flat multilayer perceptron over the same features would isolate how much the graph structure contributes. The same interface can then be taken to real tick data, with a richer transaction-cost model and explicit latency and slippage, where I expect little exploitable arbitrage and the cost-aware and directional components to dominate. The reward can be made risk-adjusted to discourage the uncompensated directional risk that the decomposition exposes. The longer-term goal is open-currency-position management, where a graph-attention agent could help a desk or treasury allocate and hedge exposure under cost.

The contribution is ultimately methodological. This paper offers a graph-based reinforcement-learning framework for FX, a currency graph carrying executable-rate and arbitrage edge features, a graph-attention encoder, and an explicit arbitrage-execution action, together with a controlled, oracle-bounded testbed that makes claims about representation learning falsifiable. It is this combination, and the honest account of what the representation does and does not enable, that this paper contributes to the study of learned state representations for financial decision-making.

References

- [1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [2] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*, 2018.
- [3] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2016.
- [4] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International Conference on Machine Learning (ICML)*, 2014.
- [5] Daniel J. Fenn, Sam D. Howison, Mark McDonald, Stacy Williams, and Neil F. Johnson. The mirage of triangular arbitrage in the spot foreign exchange market. *International Journal of Theoretical and Applied Finance*, 12(8):1105–1123, 2009.
- [6] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, 2017.
- [7] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- [8] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [9] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [10] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022.
- [11] John Moody and Matthew Saffell. Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 12(4):875–889, 2001.
- [12] Yue Deng, Feng Bao, Youyong Kong, Zhiquan Ren, and Qionghai Dai. Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3):653–664, 2017.

- [13] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017.
- [14] Zhipeng Liang, Hao Chen, Junhao Zhu, Kangkang Jiang, and Yanran Li. Adversarial deep reinforcement learning in portfolio management. *arXiv preprint arXiv:1808.09940*, 2018.
- [15] Yun-Cheng Tsai and Chun-Chieh Wang. Deep reinforcement learning for foreign exchange trading. *arXiv preprint arXiv:1908.08036*, 2019.
- [16] Fuli Feng, Xiangnan He, Xiang Wang, Cheng Luo, Yiqun Liu, and Tat-Seng Chua. Temporal relational ranking for stock prediction. *ACM Transactions on Information Systems*, 37(2), 2019.
- [17] Si Shi, Jianjun Li, Guohui Li, Peng Pan, Qi Chen, and Qing Sun. GPM: A graph convolutional network based reinforcement learning framework for portfolio management. *Neurocomputing*, 498, 2022.
- [18] Robert F. Engle and Clive W. J. Granger. Co-integration and error correction: Representation, estimation, and testing. *Econometrica*, 55(2):251–276, 1987.
- [19] Yoonsik Hong and Diego Klabjan. Graph learning for foreign exchange rate prediction and statistical arbitrage. *arXiv preprint arXiv:2508.14784*, 2025.
- [20] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [21] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [22] Mark Towers et al. Gymnasium, 2023. Farama Foundation.

A Reproducibility and hyperparameters

All randomness is seeded (Python, NumPy, PyTorch, and the environment and SB3 seeds), and the full data-generating configuration is hashed and stored alongside each saved model, so a trained agent maps unambiguously to its data; loading a configuration whose schema has drifted raises a warning. The implementation is in Python with PyTorch, PyTorch Geometric [21], Gymnasium [22], and Stable-Baselines3 [20], and is covered by an automated test suite. Table 4 lists the main hyperparameters.

Table 4: Main hyperparameters.

Group	Parameter	Value
Data	currencies N / ticks / window W	4 / 5000 / 20
	train seed / test seed	42 / 1
Env	initial NAV	\$10,000
	reward scale κ / turnover λ	100 / 0.005
	softmax temperature τ	3
	max drawdown / penalty Π	0.5 / 1.0
GAT	node $\rightarrow$ hidden $\times$ heads	$3 \rightarrow 16 \times 4$
	hidden $\rightarrow$ out	$64 \rightarrow 32$
	embedding dim	128
TD3	actor/critic arch	[128, 64]
	learning rate / batch / buffer	10^{-4} / 256 / 10^5
	discount γ / action noise	0.99 / 0.15
	training steps	6×10^4

B Notation

Table 5: Notation. (The GAT attention coefficient α_{ij} in (4) is distinct from the ask quote $\alpha_t(i, j)$.)

Symbol	Meaning
$\mathcal{C}, N$	set of currencies; their number ($N = 4$)
t, W	tick index; trailing window length
$\beta_t(i, j), \alpha_t(i, j)$	executable bid / ask for converting $i \rightarrow j$
$\text{conv}_t(x, i, j)$	spread-aware conversion of x from i to j
$b_t^i, v_t^i, V_t, \omega_t^i$	balance, USD value, NAV, weight of currency i
$\mathcal{G}_t, \mathcal{E}$	state graph; its directed edge set
$\mathbf{x}_t^i, \mathbf{z}_t^{ij}$	node features; edge features
$\mathbf{a}_t, \mathbf{w}_t, \eta_t, \tau$	action; target weights; arb. intensity; temperature
$\mathcal{T}, \delta, \phi_t(\delta)$	directed triangles; a cycle; its cycle factor
$r_t, \kappa, \lambda, \rho_t, \gamma$	reward; scale; turnover penalty; turnover; discount
α_{ij}	GAT attention coefficient